République tunisienne
Ministère de l'Enseignement supérieur et de la Recherche scientifique
TEK-UP University
École privée supérieure de technologie et d'ingénierie

Conception et Implémentation d'un Système Intelligent de Gestion d'Atelier Automobile

Présenté en vue de l'obtention de la

Cycle d'Ingénieur en Informatique

Parcours : Génie Logiciel

Réalisé par

Ihebeddine Saafi | Arbi Hazbri | Louay Lewati

MecaManage

Thème du projet

Année Universitaire : 2025–2026

Résumé

MecaRage est une plateforme logicielle complète dédiée à la gestion moderne et intelligente des ateliers mécaniques. Elle repose sur une architecture distribuée à plusieurs couches : un back-end **ASP.NET Core 9** en Clean Architecture (CQRS et MediatR), un front-end **Angular 21** avec Tailwind CSS, un service d'intelligence artificielle fondé sur **FastAPI** et **Ollama** (LLM local) pour le diagnostic contextuel par RAG (Retrieval-Augmented Generation), et un module mobile **Android** (Java) utilisant **SQLite** pour le stockage local et un service **FastAPI** de synchronisation directe vers MySQL.

La plateforme couvre l'intégralité du cycle de vie d'une intervention mécanique : dépôt des symptômes, diagnostic IA, prise de rendez-vous, affectation des mécaniciens, rapport d'examen, facturation, approbation client, réparation, contrôle qualité et paiement — le tout tracé par un suivi de cycle de vie unifié (*Intervention Lifecycle Tracker*).

Mots-clés : Gestion d'atelier, Clean Architecture, CQRS, Angular 21, ASP.NET Core 9, SQLite, FastAPI, Intelligence Artificielle, RAG, Android, MySQL.

Abstract

MecaRage is a comprehensive software platform for the modern and intelligent management of mechanical workshops. It is built on a multi-layer distributed architecture : an **ASP.NET Core 9** back-end with Clean Architecture (CQRS and MediatR), an **Angular 21** front-end with Tailwind CSS, an AI service based on **FastAPI** and **Ollama** (local LLM) for RAG-based contextual diagnosis, and an **Android** mobile module (Java) integrating **Firebase** for real-time notifications.

The platform covers the complete lifecycle of a mechanical intervention : symptom submission, AI diagnosis, appointment scheduling, mechanic assignment, examination report, invoicing, client approval, repair, quality validation and payment — all tracked by a unified Intervention Lifecycle Tracker.

Keywords : Workshop management, Clean Architecture, CQRS, Angular 21, ASP.NET Core 9, SQLite, FastAPI, Artificial Intelligence, RAG, Android, MySQL.

Table des matières

Résumé **i**

Abstract **ii**

Table des matières **iv**

Table des figures **v**

Liste des tableaux **vi**

1 Les Bases **1**

1.1 Introduction 1

1.2 PrÃ©sentation de l'entreprise 1

1.3 PrÃ©sentation du projet 1

1.3.1 ProblÃ©matique 1

1.3.2 Ã‰tude de l'existant 2

1.3.3 Solution proposÃ©e 2

1.4 Conclusion 3

2 SpÃ©cification Fonctionnelle et Technique **4**

2.1 Introduction 4

2.2 SpÃ©cification Fonctionnelle 4

2.2.1 Exigences Fonctionnelles 4

2.2.2 Diagramme de cas d'utilisation global 6

2.2.3 Diagramme de classes global 7

2.2.4 Backlog Produit 7

2.3 SpÃ©cification Technique 9

2.3.1 Exigences Non-Fonctionnelles 9

2.3.2 Outils et Environnement de DÃ©veloppement 10

2.4 Conclusion 10

3 Sprint 1 : Authentification et Gestion des Profils **11**

3.1 Introduction 11

3.2 Identification du Backlog Sprint 1 11

3.3 Conception 12

3.3.1 Diagramme de cas d'utilisation — Sprint 1 12

3.3.2 Diagramme de classes — Sprint 1 13

3.3.3 Diagramme de sÃ©quence 13
3.4 Conclusion 14

4 Sprint 2 : Gestion des DonnÃ©es de RÃ©fÃ©rence 16
4.1 Introduction 16
4.2 Identification du Backlog Sprint 2 16
4.3 Conception 17
4.3.1 Diagramme de cas d'utilisation — Sprint 2 17
4.3.2 Diagramme de classes — Sprint 2 18
4.3.3 Diagramme de sÃ©quence 18
4.4 Conclusion 19

5 Sprint 3 : Rapports de SymptÃ´mes, Diagnostic IA et Rendez-vous 20
5.1 Introduction 20
5.2 Identification du Backlog Sprint 3 20
5.3 Conception 21
5.3.1 Diagramme de cas d'utilisation — Sprint 3 21
5.3.2 Pipeline du Diagnostic IA 23
5.3.3 Diagramme de classes — Sprint 3 24
5.3.4 Diagramme de sÃ©quence 24
5.4 Conclusion 25

6 Sprint 4 : OpÃ©rations d'Atelier et Cycle de Vie des Interventions 26
6.1 Introduction 26
6.2 Identification du Backlog Sprint 4 27
6.3 Conception 28
6.3.1 Diagramme de cas d'utilisation — Sprint 4 28
6.3.2 Diagramme de classes — Sprint 4 30
6.3.3 Diagramme de sÃ©quence 30
6.4 Conclusion 31

Conclusion Générale 33

Bibliographie 35

Table des figures

1.1 Architecture globale de la plateforme MecaRage 3

2.1 Diagramme de cas d'utilisation global — MecaRage 6
2.2 Diagramme de classes global — EntitÃ©s du domaine MecaRage . . . 7

3.1 Diagramme de cas d'utilisation — Sprint 1 12
3.2 Diagramme de classes — Sprint 1 (Authentification) 13
3.3 Diagramme de sÃ©quence SC1 — Connexion (Login) 14

4.1 Diagramme de cas d'utilisation — Sprint 2 17
4.2 Diagramme de classes — Sprint 2 (DonnÃ©es de RÃ©fÃ©rence) 18
4.3 Diagramme de sÃ©quence SC3 — CrÃ©ation d'un Garage 19

5.1 Diagramme de cas d'utilisation — Sprint 3 21
5.2 Pipeline RAG — Service de diagnostic IA 23
5.3 Diagramme de classes — Sprint 3 (SymptÃ´mes, IA et Rendez-vous) . 24
5.4 Diagramme de sÃ©quence SC5 — Soumission de SymptÃ´mes et Diagnostic IA 25

6.1 Diagramme de cas d'utilisation — Sprint 4 28
6.2 Machine d'Ã©tats — Intervention Lifecycle Tracker 29
6.3 Diagramme de classes — Sprint 4 (Atelier et Cycle de Vie) 30
6.4 Diagramme de sÃ©quence SC6 — Finalisation et Approbation du Devis 31

Liste des tableaux

1.1 ProblÃ¨mes identifiÃ©s dans le systÃ¨me actuel 2
1.2 Outils existants et leurs limites 2

2.1 Exigences fonctionnelles par rÃ´le 4
2.2 Backlog Produit complet — MecaRage 7
2.3 Exigences non-fonctionnelles 9
2.4 Stack technologique de MecaRage 10

3.1 Backlog Sprint 1 — Authentification 11

4.1 Backlog Sprint 2 — DonnÃ©es de RÃ©fÃ©rence 16

5.1 Backlog Sprint 3 — SymptÃ´mes, IA et Rendez-vous 20

6.1 Backlog Sprint 4 — OpÃ©rations d'Atelier 27
6.2 Récapitulatif des réalisations par sprint 33

Chapitre 1

Les Bases

*Ce chapitre prÃ©sente le contexte gÃ©nÃ©ral du projet **MecaRage** : la structure de l'entreprise commanditaire, la problÃ©matique Ã l'origine du projet et la solution proposÃ©e.*

1.1 Introduction

Dans un contexte de digitalisation croissante des mÃ©tiers de la maintenance automobile, les ateliers mÃ©caniques ressentent une pression de plus en plus forte pour moderniser leurs processus. MecaRage a Ã©tÃ© conÃ§u pour rÃ©pondre Ã ce besoin en offrant une solution numÃ©rique complÃ¨te, couvrant l'ensemble de la chaÃ®ne opÃ©rationnelle d'un rÃ©seau d'ateliers multi-sites.

1.2 PrÃ©sentation de l'entreprise

L'entreprise partenaire coordonne plusieurs *garages* rÃ©partis gÃ©ographiquement, chacun disposant d'une Ã©quipe de mÃ©caniciens supervisÃ©e par un chef d'atelier. La direction centrale assure la supervision globale des opÃ©rations et de la facturation.

1.3 PrÃ©sentation du projet

1.3.1 ProblÃ©matique

La gestion d'un rÃ©seau d'ateliers mÃ©caniques implique une coordination complexe entre plusieurs parties prenantes : les clients qui dÃ©posent leurs vÃ©hicules, les mÃ©caniciens qui exÃ©cutent les travaux, le chef d'atelier qui orchestre les ressources, et la direction qui supervise les finances.

Domaine	ProblÃ¨me	ConsÃ©quence
Communication	RDV par tÃ©lÃ©phone/WhatsApp	Pas de traÃ§abilitÃ©
Rapports	Fiches papier mÃ©canicien	Risque de perte, illisibilitÃ©
Facturation	Tableur Excel par garage	Pas de vision consolidÃ©e
Suivi	Aucun outil de statut en temps rÃ©el	Manque de visibilitÃ© client
Diagnostic	RÃ©ception sans prÃ©-analyse	Perte de temps Ã l'accueil
Stock	Inventaire manuel piÃ¨ces	Ruptures non anticipÃ©es

Table 1.1. ProblÃ¨mes identifiÃ©s dans le systÃ¨me actuel

1.3.2 Ã‰tude de l'existant

L'existant se compose d'une combinaison d'outils non intÃ©grÃ©s :

Outil	Usage actuel	Limite principale
WhatsApp / SMS	Prise de RDV, communications	Informel, non traÃ§able
Excel	Inventaire piÃ¨ces, facturation	Pas de collaboration temps rÃ©el
Papier	Rapports mÃ©caniciens	Perte, archivage difficile
Tableau blanc	Suivi visuel de l'atelier	Non numÃ©rique, local uniquement

Table 1.2. Outils existants et leurs limites

1.3.3 Solution proposÃ©e

MecaRage est une plateforme numÃ©rique unifiÃ©e couvrant l'intÃ©gralitÃ© du cycle de vie d'une intervention mÃ©canique.

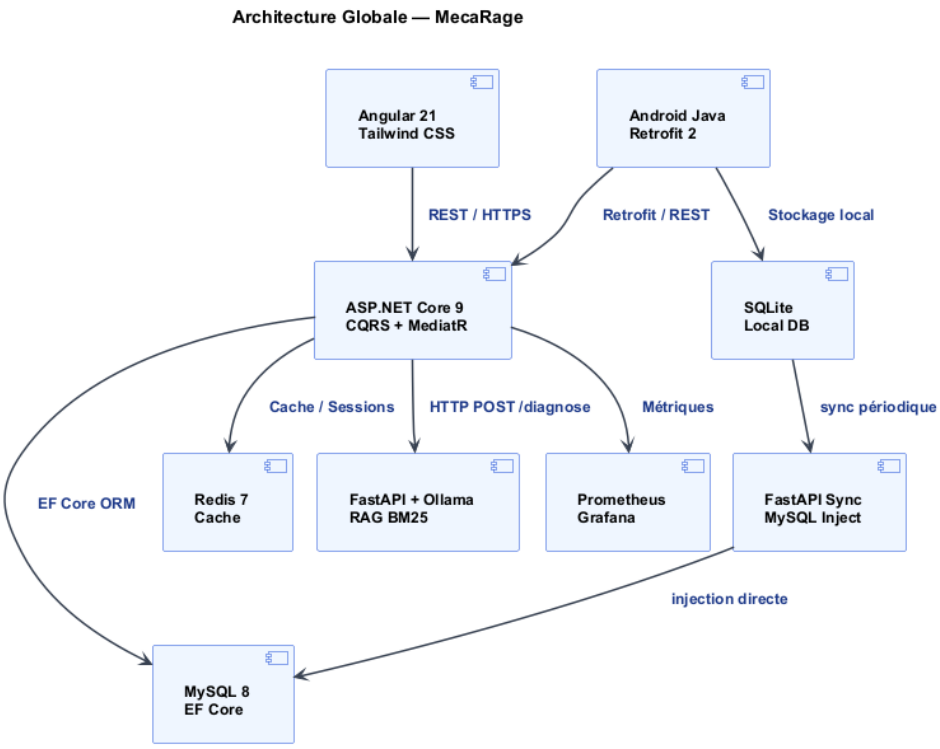


Figure 1.1. Architecture globale de la plateforme MecaRage

Les apports clÃ©s :

- **Application web** accessible par tous les rÃ´les (Angular 21).
- **Back-end robuste** (ASP.NET Core 9, Clean Architecture, CQRS).
- **Service IA** (FastAPI + Ollama) pour le diagnostic prÃ©liminaire.
- **Module mobile Android** (Java) avec SQLite local et synchronisation via FastAPI → MySQL.
- **Gestion multi-tenant** : plusieurs entreprises sur la mÃªme plateforme.
- **Intervention Lifecycle Tracker** : suivi automatique sur 10 Ã©tats.
- **GÃ©nÃ©ration PDF** des factures via QuestPDF.

1.4 Conclusion

Ce chapitre a prÃ©sentÃ© le contexte mÃ©tier qui a motivÃ© la crÃ©ation de MecaRage. Le chapitre suivant dÃ©taille les spÃ©cifications fonctionnelles et techniques.

Chapitre 2

SpÃ©cification Fonctionnelle et Technique

Ce chapitre formalise les exigences fonctionnelles et non-fonctionnelles de la plateforme Ã travers le diagramme de cas d'utilisation global, le diagramme de classes du domaine, le backlog produit complet et les choix techniques.

2.1 Introduction

La phase de spÃ©cification garantit que l'Ã©quipe de dÃ©veloppement et les parties prenantes partagent une vision commune des fonctionnalitÃ©s Ã rÃ©aliser. Ce chapitre prÃ©sente les outils de modÃ©lisation utilisÃ©s pour formaliser cette vision.

2.2 SpÃ©cification Fonctionnelle

2.2.1 Exigences Fonctionnelles

Table 2.1. Exigences fonctionnelles par rÃ´le

NÂ°	Acteur	Exigence
EF01	Tous	S'authentifier avec email et mot de passe (JWT)
EF02	Tous	RafraÃ®chir automatiquement le token d'accÃ¨s
EF03	Tous	Modifier son mot de passe
EF04	SuperAdmin	CrÃ©er et gÃ©rer les tenants (entreprises)
EF05	AdminEntreprise	CrÃ©er et gÃ©rer les garages du tenant
EF06	AdminEntreprise	CrÃ©er et gÃ©rer le personnel (chefs, mÃ©caniciens)

NÂ°	Acteur	Exigence
EF07	AdminEntreprise	Consulter les indicateurs de performance (KPI)
EF08	AdminEntreprise	Enregistrer le paiement d'une intervention
EF09	Client	S'inscrire sur la plateforme
EF10	Client	GÃ©rer ses vÃ©hicules (ajout, consultation)
EF11	Client	Soumettre un rapport de symptÃ´mes textuels
EF12	SystÃ¨me	GÃ©nÃ©rer un diagnostic IA (Ollama RAG) automatiquement
EF13	Client	RÃ©server un rendez-vous dans la plage proposÃ©e
EF14	Client	Approuver ou refuser un devis
EF15	Client	Suivre le cycle de vie de son intervention
EF16	Client	TÃ©lÃ©charger sa facture en PDF
EF17	ChefAtelier	RÃ©viser les rapports et proposer une plage RDV
EF18	ChefAtelier	Approuver ou refuser des rendez-vous
EF19	ChefAtelier	Assigner des mÃ©caniciens Ã une tÃ¢che de rÃ©paration
EF20	ChefAtelier	RÃ©viser le rapport d'examen du mÃ©canicien
EF21	ChefAtelier	CrÃ©er et finaliser une facture (devis)
EF22	ChefAtelier	Valider la rÃ©paration et notifier le client
EF23	MÃ©canicien	Consulter ses tÃ¢ches assignÃ©es
EF24	MÃ©canicien	Soumettre un rapport d'examen (piÃ¨ces, observations)
EF25	MÃ©canicien	Soumettre un rapport de fin de rÃ©paration
EF26	SystÃ¨me	Envoyer notifications in-app (MySQL) et push (FCM)
EF27	AdminEntreprise	GÃ©rer l'inventaire des piÃ¨ces dÃ©tachÃ©es
EF28	Tous	AccÃ©der via application Android (client et mÃ©canicien)

2.2.2 Diagramme de cas d'utilisation global

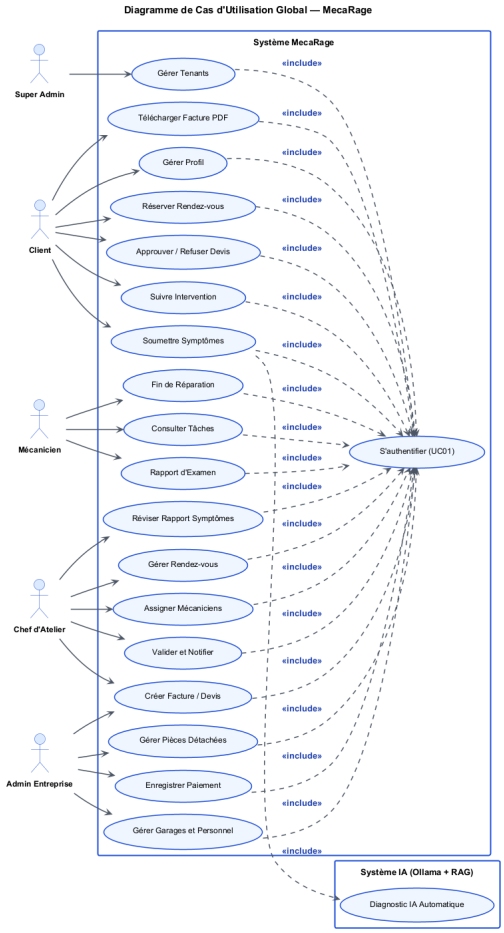


Figure 2.1. Diagramme de cas d'utilisation global — MecaRage

2.2.3 Diagramme de classes global

Figure 2.2. Diagramme de classes global — EntitÃ©s du domaine MecaRage

2.2.4 Backlog Produit

Table 2.2. Backlog Produit complet — MecaRage

ID	En tant que	Je veux	PrioritÃ	SP	Sprint
US01	Utilisateur	M'inscrire avec email et mot de passe	**Haute**	3	Sprint 1
US02	Utilisateur	Me connecter et recevoir un JWT	**Haute**	3	Sprint 1
US03	Utilisateur	RafraÃ®chir mon token automatiquement	**Haute**	2	Sprint 1
US04	Utilisateur	Modifier mon mot de passe	**Moyenne**	2	Sprint 1
US05	SuperAdmin	CrÃ©er et gÃ©rer les tenants	**Haute**	3	Sprint 2

ID	En tant que	Je veux	PrioritÃ	SP	Sprint
US06	AdminEntreprise	CrÃ©er et gÃ©rer des garages	**Haute**	5	Sprint 2
US07	AdminEntreprise	GÃ©rer le personnel	**Haute**	5	Sprint 2
US08	Client	Enregistrer et gÃ©rer ses vÃ©hicules	**Haute**	3	Sprint 2
US09	AdminEntreprise	GÃ©rer l'inventaire des piÃ¨ces	**Moyenne**	5	Sprint 2
US10	Client	Soumettre un rapport de symptÃ´mes	**Haute**	5	Sprint 3
US11	SystÃ¨me	GÃ©nÃ©rer un diagnostic IA (RAG)	**Haute**	8	Sprint 3
US12	ChefAtelier	RÃ©viser les rapports et proposer une plage RDV	**Haute**	5	Sprint 3
US13	Client	RÃ©server un rendez-vous	**Haute**	3	Sprint 3
US14	ChefAtelier	Approuver ou refuser un rendez-vous	**Haute**	3	Sprint 3
US15	ChefAtelier	Assigner des mÃ©caniciens Ã une tÃ¢che	**Haute**	5	Sprint 4
US16	MÃ©canicien	Consulter ses tÃ¢ches assignÃ©es	**Haute**	3	Sprint 4
US17	MÃ©canicien	Soumettre un rapport d'examen	**Haute**	5	Sprint 4
US18	ChefAtelier	RÃ©viser et valider le rapport d'examen	**Haute**	3	Sprint 4
US19	ChefAtelier	CrÃ©er et finaliser une facture	**Haute**	5	Sprint 4
US20	Client	Approuver ou refuser un devis	**Haute**	3	Sprint 4
US21	MÃ©canicien	Soumettre la fin de rÃ©paration	**Haute**	3	Sprint 4
US22	ChefAtelier	Valider et notifier le client	**Haute**	3	Sprint 4
US23	AdminEntreprise	Enregistrer le paiement	**Haute**	3	Sprint 4

ID	En tant que	Je veux	PrioritÃ	SP	Sprint
US24	SystÃ¨me	Tracer le cycle de vie automatiquement	Haute	8	Sprint 4
US25	Utilisateurs	Recevoir notifications in-app et push	Moyenne	5	Sprint 4
US26	Client	TÃ©lÃ©charger la facture en PDF	Moyenne	3	Sprint 4
US27	Client/MÃ©canicien	Utiliser l'application Android native	Moyenne	13	Sprint 4
			Total Story Points	**104 SP**	

2.3 SpÃ©cification Technique

2.3.1 Exigences Non-Fonctionnelles

CritÃ¨re	Exigence
Performance	Temps de rÃ©ponse API $<$ 300 ms pour 95% des requÃªtes
SÃ©curitÃ©	JWT HS256, hachage bcrypt, isolation multi-tenant, CORS contrÃ´lÃ©
ScalabilitÃ©	Architecture stateless ; front-end dÃ©couplÃ© ; services indÃ©pendants
DisponibilitÃ©	Docker Compose avec healthchecks et `restart: unless-stopped`
MaintenabilitÃ©	Clean Architecture, CQRS, tests xUnit, couverture $\geq$ 70%
CompatibilitÃ©	Android 8+ (API 26) ; navigateurs modernes (Chrome, Firefox, Edge)

Table 2.3. Exigences non-fonctionnelles

2.3.2 Outils et Environnement de DÃ©veloppement

Couche	Technologie	Version / RÃ´le
Front-end Web	Angular + Tailwind CSS	v21.1 / SPA rÃ©active, dark mode
Back-end API	ASP.NET Core + MediatR	v9.0 / API REST, CQRS, Clean Arch.
Base de donnÃ©es	MySQL + Pomelo EF Core	8.0 / ORM relationnel, migrations
Cache	Redis	7.0 / sessions, invalidation cache
Service IA	Python FastAPI + Ollama	3.13 / RAG BM25 + LLM local
Mobile	Android Java + SQLite	SDK 26+ / stockage local + sync FastAPI → MySQL
PDF	QuestPDF	2026.x / gÃ©nÃ©ration factures
DÃ©ploiement	Docker + Docker Compose	25+ / containerisation totale
Monitoring	Prometheus + Grafana	— / mÃ©triques et alertes
Gestion projet	MÃ©thode Agile Scrum	4 sprints, 2 semaines/sprint

Table 2.4. Stack technologique de MecaRage

2.4 Conclusion

Ce chapitre a formalisÃ© les besoins de la plateforme par 27 user stories organisÃ©es en 4 sprints (104 story points au total). Les chapitres suivants dÃ©taillent la conception et le dÃ©veloppement de chaque sprint.

Chapitre 3

Sprint 1 : Authentification et Gestion des Profils

Ce premier sprint pose les fondations de la plateforme en implÃ©mentant le systÃ¨me d'authentification sÃ©curisÃ© par JWT et la gestion des profils. Il constitue le prÃ©requis de toutes les fonctionnalitÃ©s ultÃ©rieures.

3.1 Introduction

L'authentification est le socle de toute plateforme multi-utilisateurs. Elle garantit que chaque acteur accÃ¨de uniquement aux ressources qui lui sont attribuÃ©es selon son rÃ´le. Ce sprint implÃ©mente un systÃ¨me JWT stateless, adaptÃ© au dÃ©ploiement multi-tenant et Ã l'architecture dÃ©couplÃ©e front-end/back-end.

3.2 Identification du Backlog Sprint 1

ID	User Story	PrioritÃ©	SP
US01	En tant qu'utilisateur, je veux m'inscrire avec email et mot de passe	**Haute**	3
US02	En tant qu'utilisateur, je veux me connecter et recevoir un JWT	**Haute**	3
US03	En tant qu'utilisateur, je veux que mon token se rafraÃ®chisse automatiquement	**Haute**	2
US04	En tant qu'utilisateur, je veux modifier mon mot de passe	**Moyenne**	2
		Total Sprint 1	**10 SP**

Table 3.1. Backlog Sprint 1 — Authentification

TÃ¢ches techniques :

1. ModÃ¨les de domaine `User` et `Tenant`.

2. Commandes CQRS : `RegisterCommand`, `LoginCommand`, `RefreshTokenCommand`, `ChangePasswordCommand`.
3. Middleware JWT et filtre global multi-tenant EF Core.
4. Endpoints REST dans `AuthController`.
5. Ã‰crans Angular : `LoginComponent`, `SignupComponent`.
6. Intercepteur HTTP pour injection automatique du Bearer Token.

3.3 Conception

3.3.1 Diagramme de cas d'utilisation — Sprint 1

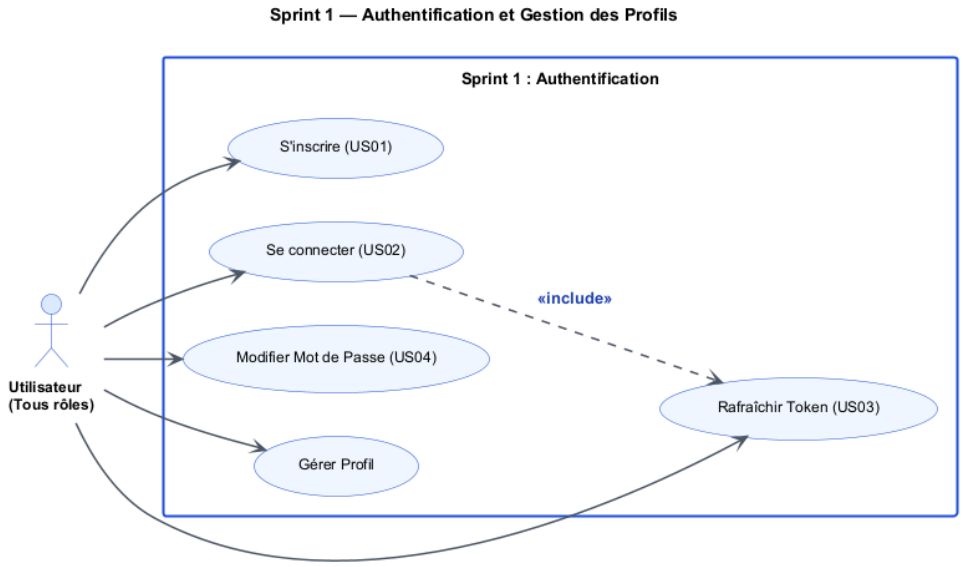


Figure 3.1. Diagramme de cas d'utilisation — Sprint 1

3.3.2 Diagramme de classes — Sprint 1

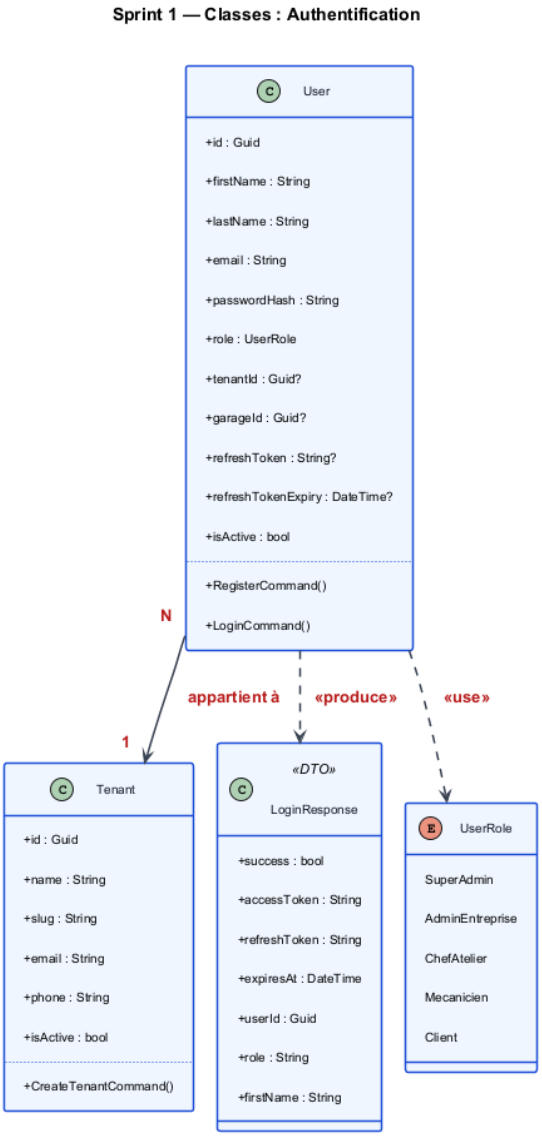


Figure 3.2. Diagramme de classes — Sprint 1 (Authentification)

3.3.3 Diagramme de sÃ©quence

SC1 — Connexion (US02) : fonctionnalitÃ© principale

L'utilisateur saisit ses identifiants. Le systÃ¨me valide, gÃ©nÃ¨re un JWT et un refresh token, les stocke en base, et les retourne au client.

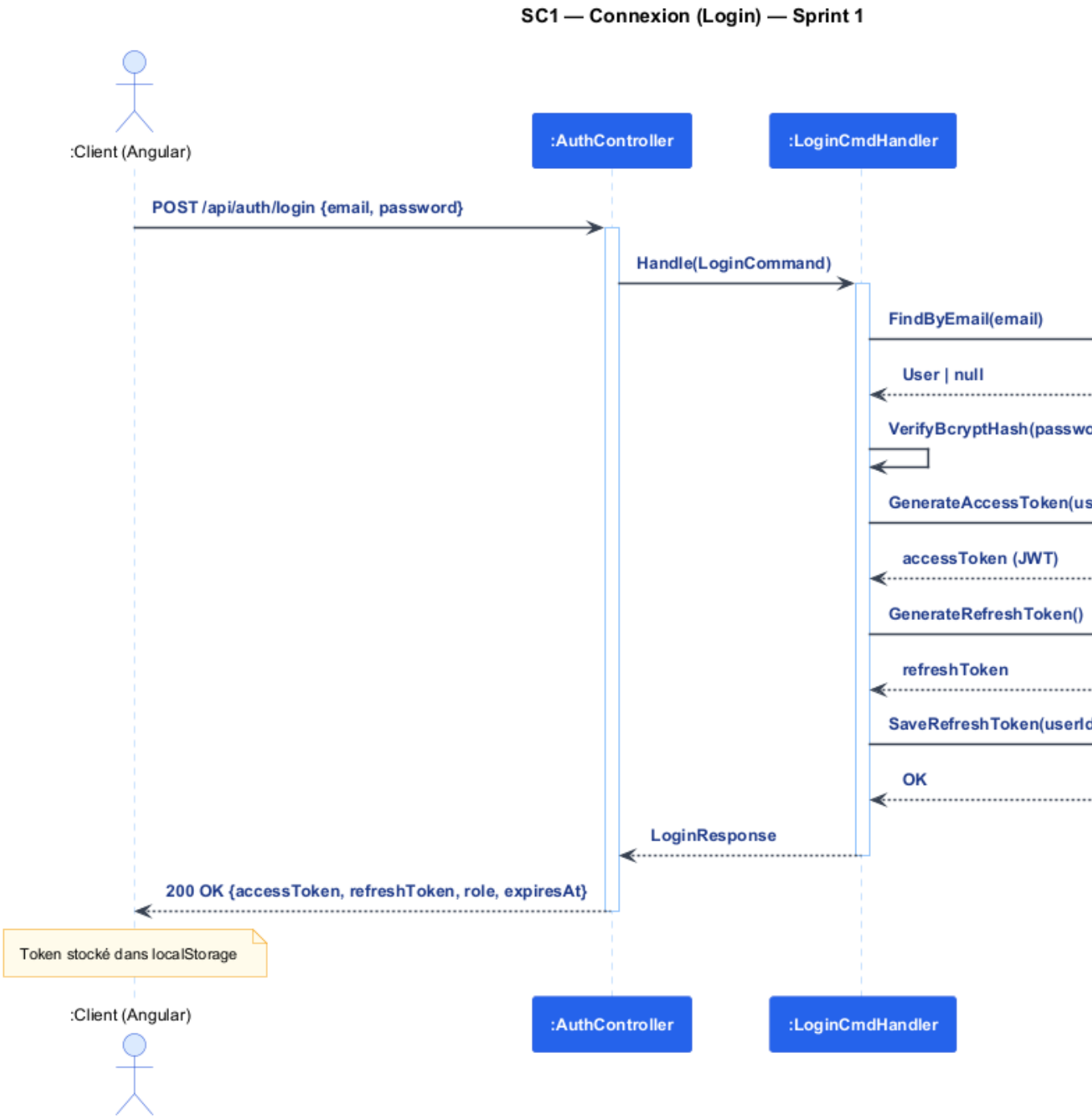


Figure 3.3. Diagramme de sÃ©quence SC1 — Connexion (Login)

3.4 Conclusion

Le Sprint 1 a Ã©tabli l'infrastructure d'authentification complÃ¨te de MecaRage. Le systÃ¨me JWT est opÃ©rationnel avec gestion du refresh token, isolation multi-tenant par filtre global EF Core (`HasQueryFilter`), et hachage BCrypt des mots de passe. Les interfaces Angular de connexion et d'inscription sont dÃ©ployÃ©es avec support du mode sombre (dark mode Tailwind CSS).

Le Sprint 1 a Ã©tabli l'infrastructure d'authentification complÃ¨te de MecaRage. Le systÃ¨me JWT est opÃ©rationnel avec gestion du refresh token, isolation multi-tenant par filtre global EF Core (`HasQueryFilter`), et hachage BCrypt des mots de passe. Les interfaces Angular de connexion et d'inscription sont dÃ©ployÃ©es avec support du mode sombre (dark mode Tailwind CSS).

Chapitre 4

Sprint 2 : Gestion des DonnÃ©es de RÃ©fÃ©rence

Ce sprint met en place les donnÃ©es fondamentales : tenants, garages, personnel, vÃ©hicules et inventaire des piÃ¨ces dÃ©tachÃ©es.

4.1 Introduction

Les donnÃ©es de rÃ©fÃ©rence constituent le socle mÃ©tier sans lequel aucune opÃ©ration n'est possible : un garage doit exister pour accueillir des rendez-vous, un vÃ©hicule doit Ãªtre enregistrÃ© pour soumettre des symptÃ´mes, et les piÃ¨ces dÃ©tachÃ©es doivent Ãªtre rÃ©fÃ©rencÃ©es pour Ãªtre incluses dans les factures.

4.2 Identification du Backlog Sprint 2

ID	User Story	PrioritÃ©	SP
US05	SuperAdmin : crÃ©er et gÃ©rer les tenants	**Haute**	3
US06	AdminEntreprise : crÃ©er et gÃ©rer des garages	**Haute**	5
US07	AdminEntreprise : gÃ©rer le personnel (chefs, mÃ©caniciens)	**Haute**	5
US08	Client : enregistrer ses vÃ©hicules	**Haute**	3
US09	AdminEntreprise : gÃ©rer l'inventaire des piÃ¨ces dÃ©tachÃ©es	**Moyenne**	5
		Total Sprint 2	**21 SP**

Table 4.1. Backlog Sprint 2 — DonnÃ©es de RÃ©fÃ©rence

TÃ¢ches techniques :

1. EntitÃ©s de domaine : `Garage`, `Vehicle`, `SparePart`.

2. Commandes CQRS : `CreateGarageCommand`, `CreateVehicleCommand`, `CreateSparePartCommar`
   `CreateStaffCommand`.
3. Migrations EF Core pour les nouvelles tables.
4. ContrÃ´leurs REST : `GaragesController`, `VehiclesController`, `SparePartsController`.
5. Politiques d'autorisation par rÃ´le (`[Authorize(Roles = ...)]`).
6. Tableau de bord administrateur Angular.

4.3 Conception

4.3.1 Diagramme de cas d'utilisation — Sprint 2

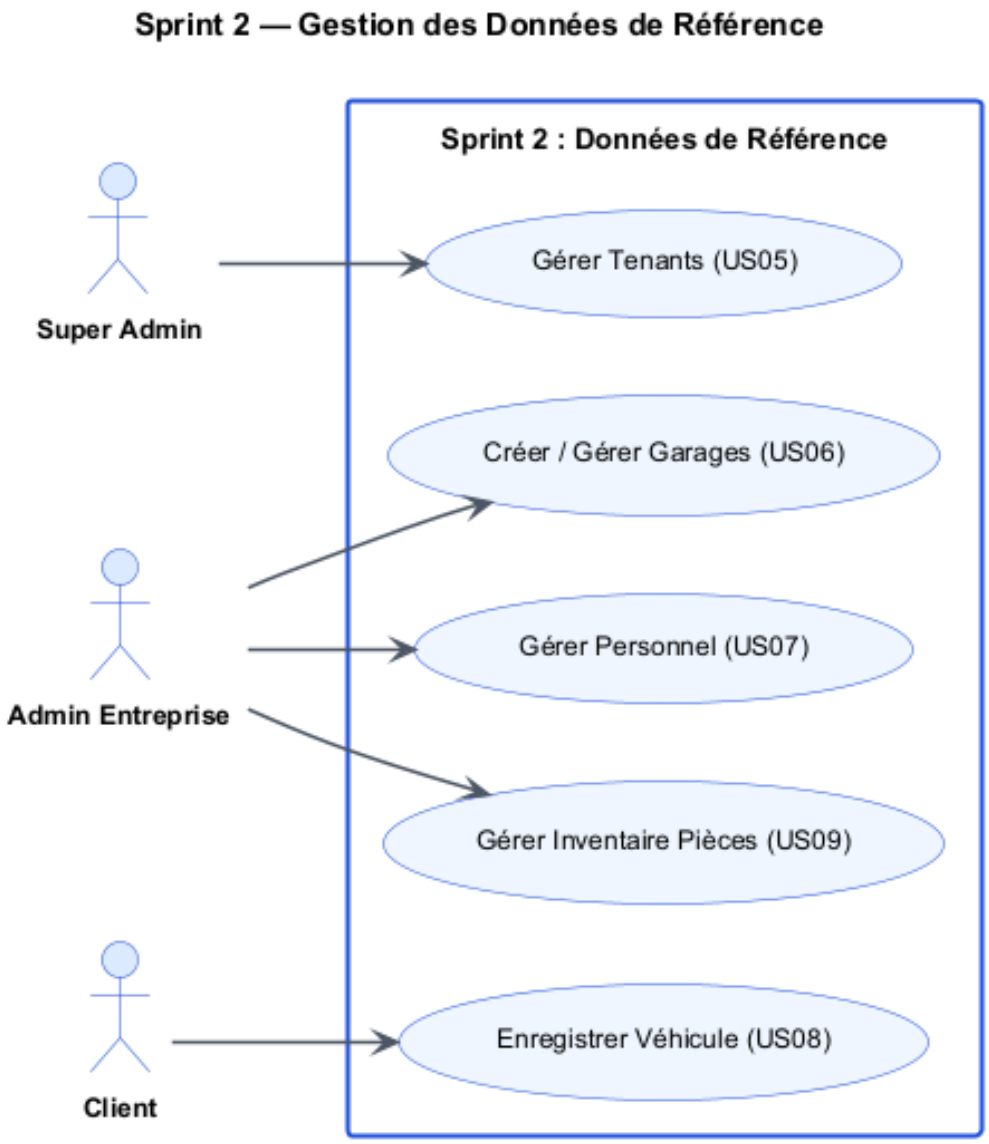


Figure 4.1. Diagramme de cas d'utilisation — Sprint 2

4.3.2 Diagramme de classes — Sprint 2

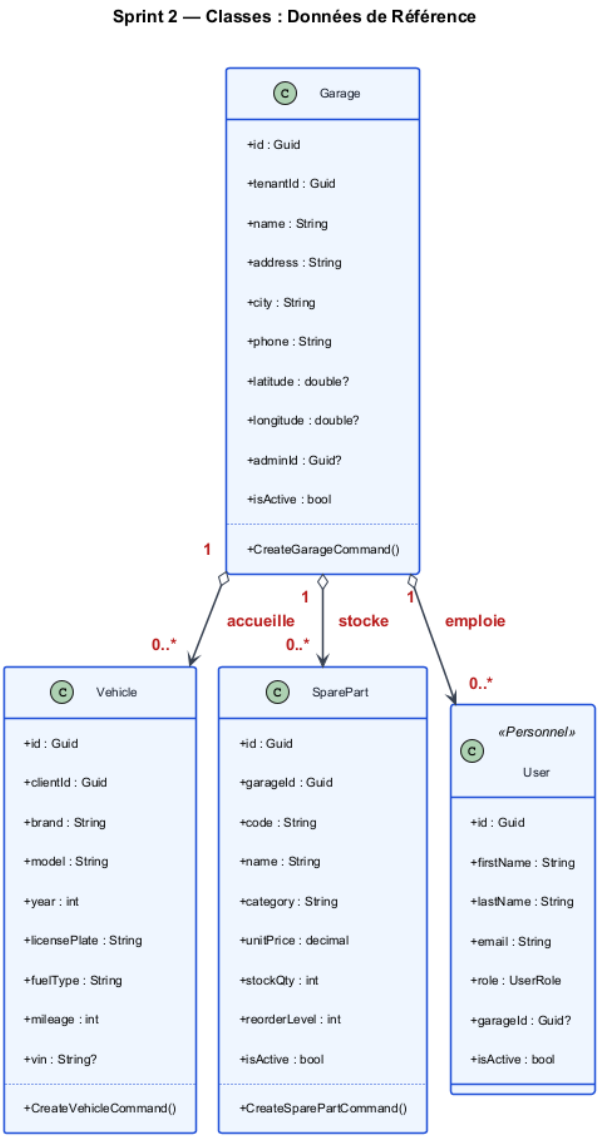


Figure 4.2. Diagramme de classes — Sprint 2 (DonnÃ©es de RÃ©fÃ©rence)

4.3.3 Diagramme de sÃ©quence

SC3 — CrÃ©ation d'un Garage (US06) : fonctionnalitÃ© principale

L'admin entreprise crÃ©e un nouveau garage aprÃ¨s vÃérification de l'existence du tenant.

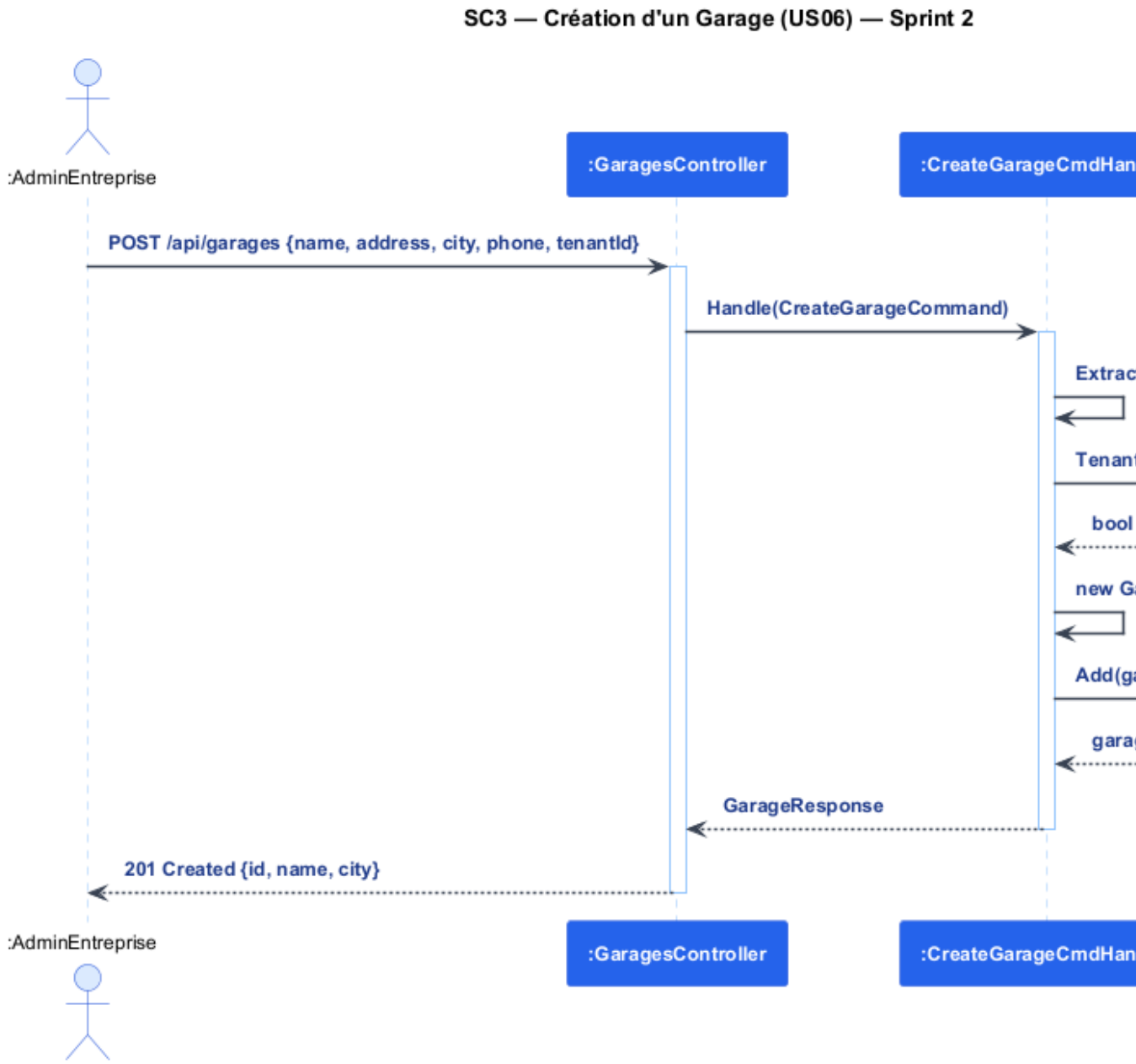


Figure 4.3. Diagramme de sÃ©quence SC3 — CrÃ©ation d'un Garage

4.4 Conclusion

Le Sprint 2 a mis en place les donnÃ©es de rÃ©fÃ©rence essentielles. Les politiques d'autorisation par rÃ´le ont Ã©tÃ© appliquÃ©es de faÃ§on granulaire : seul un `AdminEntreprise` peut crÃ©er un garage, et un `Client` ne gÃ¨re que ses propres vÃ©hicules.

Chapitre 5

Sprint 3 : Rapports de SymptÃ´mes, Diagnostic IA et Rendez-vous

Ce sprint implÃ©mente la boucle de diagnostic prÃ©liminaire et de prise de rendez-vous. Le client soumet ses symptÃ´mes, le service IA gÃ©nÃ¨re un diagnostic structurÃ©, le chef d'atelier le rÃ©vise puis propose une plage, et le client rÃ©serve son rendez-vous.

5.1 Introduction

La valeur ajoutÃ©e centrale de MecaRage rÃ©side dans son module de diagnostic IA. Le service exploite une architecture RAG (Retrieval-Augmented Generation) combinant l'algorithme BM25 pour la recherche dans la base de connaissances mÃ©canique et le modÃ¨le **Ollama** (LLM local, open-source) pour la gÃ©nÃ©ration du diagnostic structurÃ©.

5.2 Identification du Backlog Sprint 3

ID	User Story	PrioritÃ©	SP
US10	Client : soumettre une description textuelle des symptÃ´mes	**Haute**	5
US11	SystÃ¨me : dÃ©clencher automatiquement le service IA (RAG + Ollama)	**Haute**	8
US12	ChefAtelier : rÃ©viser le rapport et proposer une plage de RDV	**Haute**	5
US13	Client : rÃ©server un rendez-vous dans la plage proposÃ©e	**Haute**	3
US14	ChefAtelier : approuver ou refuser le rendez-vous	**Haute**	3
		Total Sprint 3	**24 SP**

Table 5.1. Backlog Sprint 3 — SymptÃ´mes, IA et Rendez-vous

TÃ¢ches techniques :

1. EntitÃ©s `SymptomReport` et `Appointment`.
2. Service IA FastAPI : architecture RAG BM25 + Ollama (LLM local), endpoint `POST /diagnose`.
3. Commandes : `CreateSymptomReportCommand`, `AddChefFeedbackCommand`, `CreateAppointment` `ApproveAppointmentCommand`.
4. IntÃ©gration `HttpClient` cÃ´tÃ© back-end vers le service IA.
5. Notifications automatiques au chef Ã chaque nouveau rapport.
6. Interface Angular : formulaire de symptÃ´mes, inbox chef, calendrier.

5.3 Conception

5.3.1 Diagramme de cas d'utilisation — Sprint 3

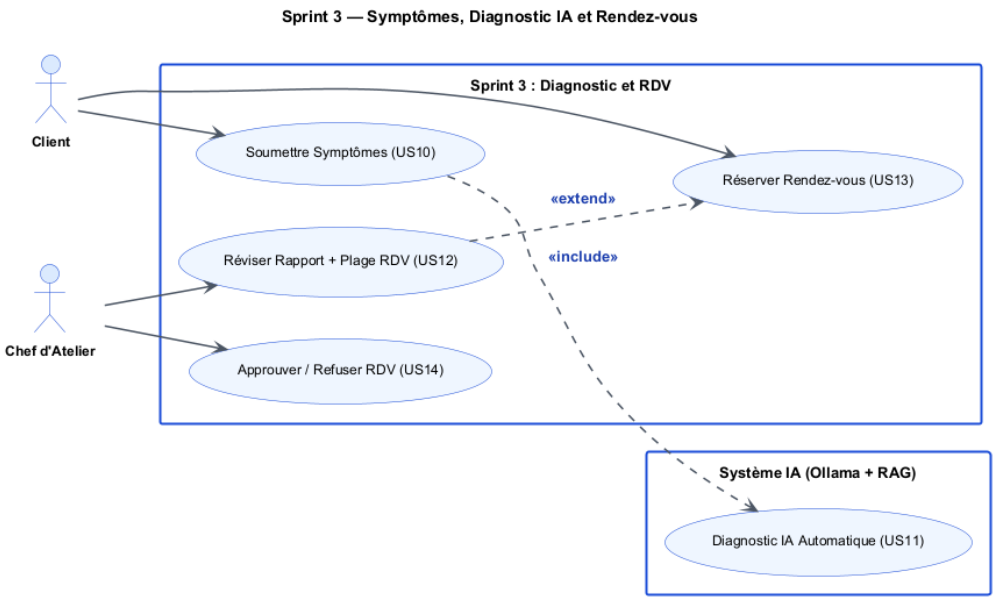
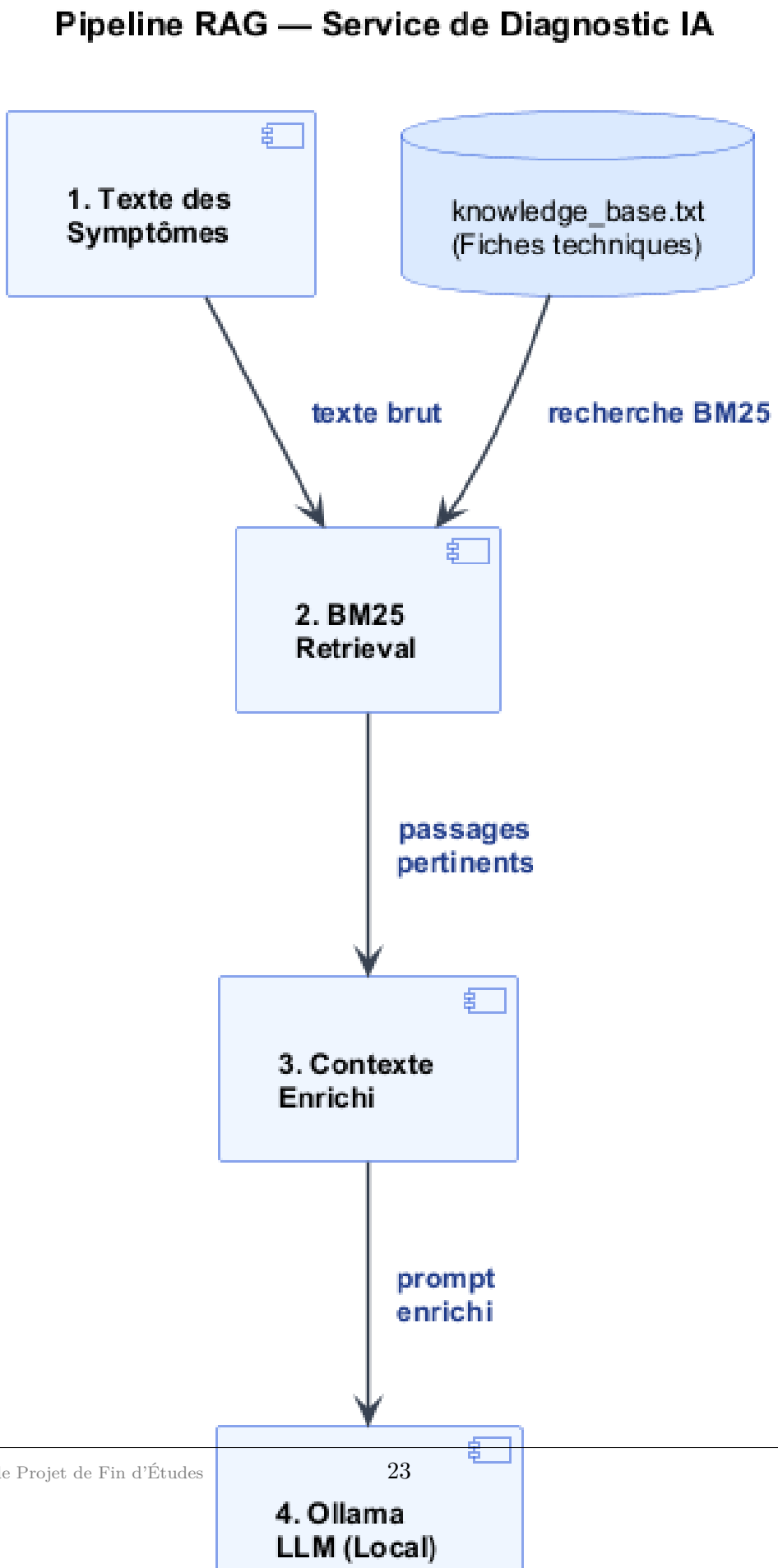


Figure 5.1. Diagramme de cas d'utilisation — Sprint 3

5.3.2 Pipeline du Diagnostic IA



5.3.3 Diagramme de classes — Sprint 3

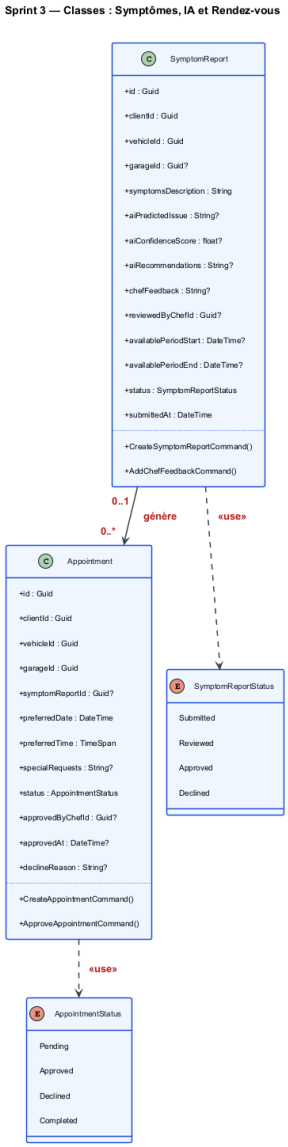


Figure 5.3. Diagramme de classes — Sprint 3 (SymptÃ´mes, IA et Rendez-vous)

5.3.4 Diagramme de sÃ©quence

SC5 — Soumission de SymptÃ´mes et Diagnostic IA (US10+US11) : fonctionnalitÃ© principale

Le client soumet des symptÃ´mes, le systÃ¨me appelle le service IA, sauvegarde le rapport enrichi, et notifie le chef d'atelier.

Figure 5.4. Diagramme de sÃ©quence SC5 — Soumission de SymptÃ´mes et Diagnostic IA

5.4 Conclusion

Le Sprint 3 a rÃ©alisÃ© la premiÃ¨re boucle de valeur visible par le client : soumettre des symptÃ´mes, recevoir un diagnostic IA instantanÃ© avec score de confiance, obtenir une plage de rendez-vous proposÃ©e par le chef, et Ãªtre notifiÃ© de sa confirmation. L'approbation d'un rendez-vous dÃ©clenche automatiquement, via MediatR, la crÃ©ation du traceur de cycle de vie de l'intervention.

Chapitre 6

Sprint 4 : OpÃ©rations d'Atelier et Cycle de Vie des Interventions

Ce sprint reprÃ©sente le cœur des opÃ©rations de l'atelier : affectation des mÃ©caniciens, examens, facturation, approbation client, rÃ©paration, validation et paiement. Toutes ces Ã©tapes sont tracÃ©es automatiquement par le ***Intervention Lifecycle Tracker*** *sur 10 Ã©tats.*

6.1 Introduction

Une fois le rendez-vous confirmÃ©, le flux opÃ©rationnel passe entre les mains de l'Ã©quipe de l'atelier. Un tracker transverse se met Ã jour automatiquement Ã chaque transition, offrant une visibilitÃ© totale sur l'Ã©tat de chaque intervention.

6.2 Identification du Backlog Sprint 4

ID	User Story	PrioritÃ©	SP
US15	Chef : assigner des mÃ©caniciens Ã une tÃ¢che	**Haute**	5
US16	MÃ©canicien : consulter ses tÃ¢ches assignÃ©es	**Haute**	3
US17	MÃ©canicien : soumettre un rapport d'examen	**Haute**	5
US18	Chef : rÃ©viser et valider le rapport d'examen	**Haute**	3
US19	Chef : crÃ©er et finaliser un devis (facture)	**Haute**	5
US20	Client : approuver ou refuser le devis	**Haute**	3
US21	MÃ©canicien : soumettre la fin de rÃ©paration	**Haute**	3
US22	Chef : valider la rÃ©paration et notifier le client	**Haute**	3
US23	Admin : enregistrer le paiement	**Haute**	3
US24	SystÃ¨me : tracer le cycle de vie (10 Ã©tats)	**Haute**	8
US25	Utilisateurs : notifications in-app et push FCM	**Moyenne**	5
US26	Client : tÃ©lÃ©charger la facture en PDF	**Moyenne**	3
US27	Client / MÃ©canicien : application Android native	**Moyenne**	13
		Total Sprint 4	**62 SP**

Table 6.1. Backlog Sprint 4 — OpÃ©rations d'Atelier

6.3 Conception

6.3.1 Diagramme de cas d'utilisation — Sprint 4

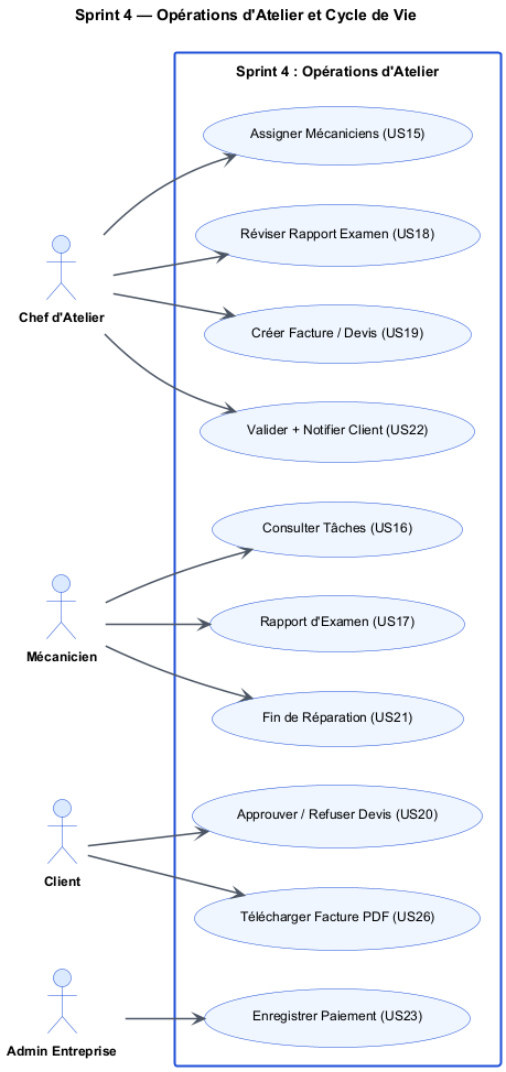
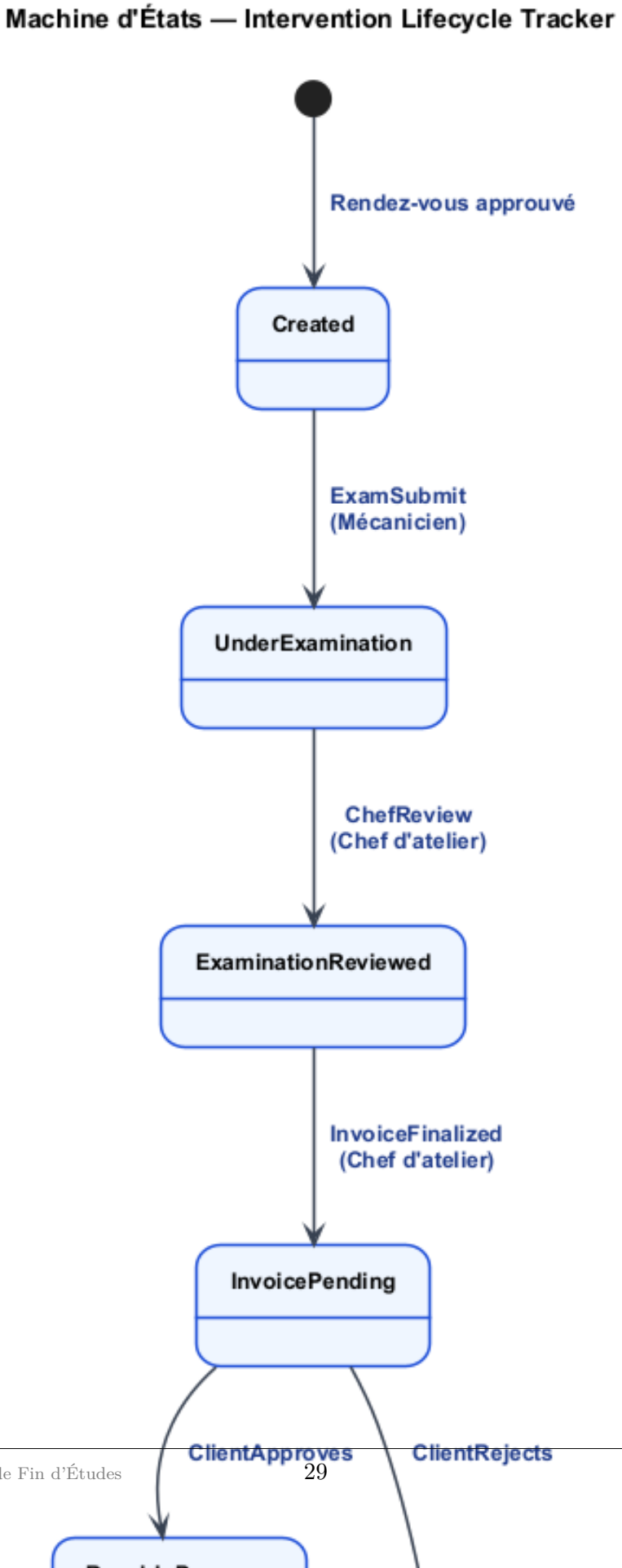


Figure 6.1. Diagramme de cas d'utilisation — Sprint 4

Machine d'Ã©tats — Intervention Lifecycle Tracker



6.3.2 Diagramme de classes — Sprint 4

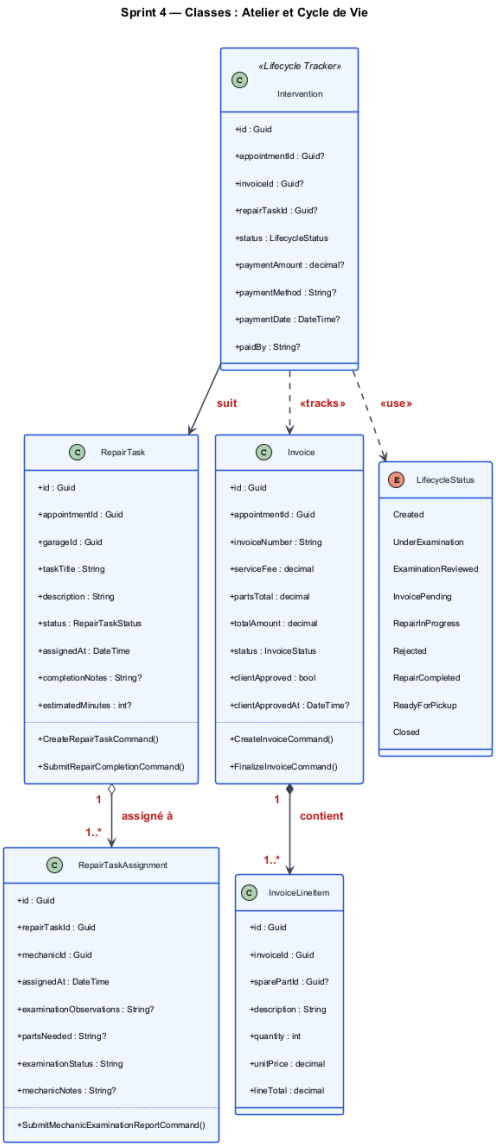


Figure 6.3. Diagramme de classes — Sprint 4 (Atelier et Cycle de Vie)

6.3.3 Diagramme de sÃ©quence

SC6 — Finalisation de Facture et Approbation Client (US19+US20) : fonctionnalitÃ© principale

Le chef finalise le devis, le client reÃ§oit une notification et l'approuve. Le tracker passe Ã `InvoicePending` puis `RepairInProgress`.

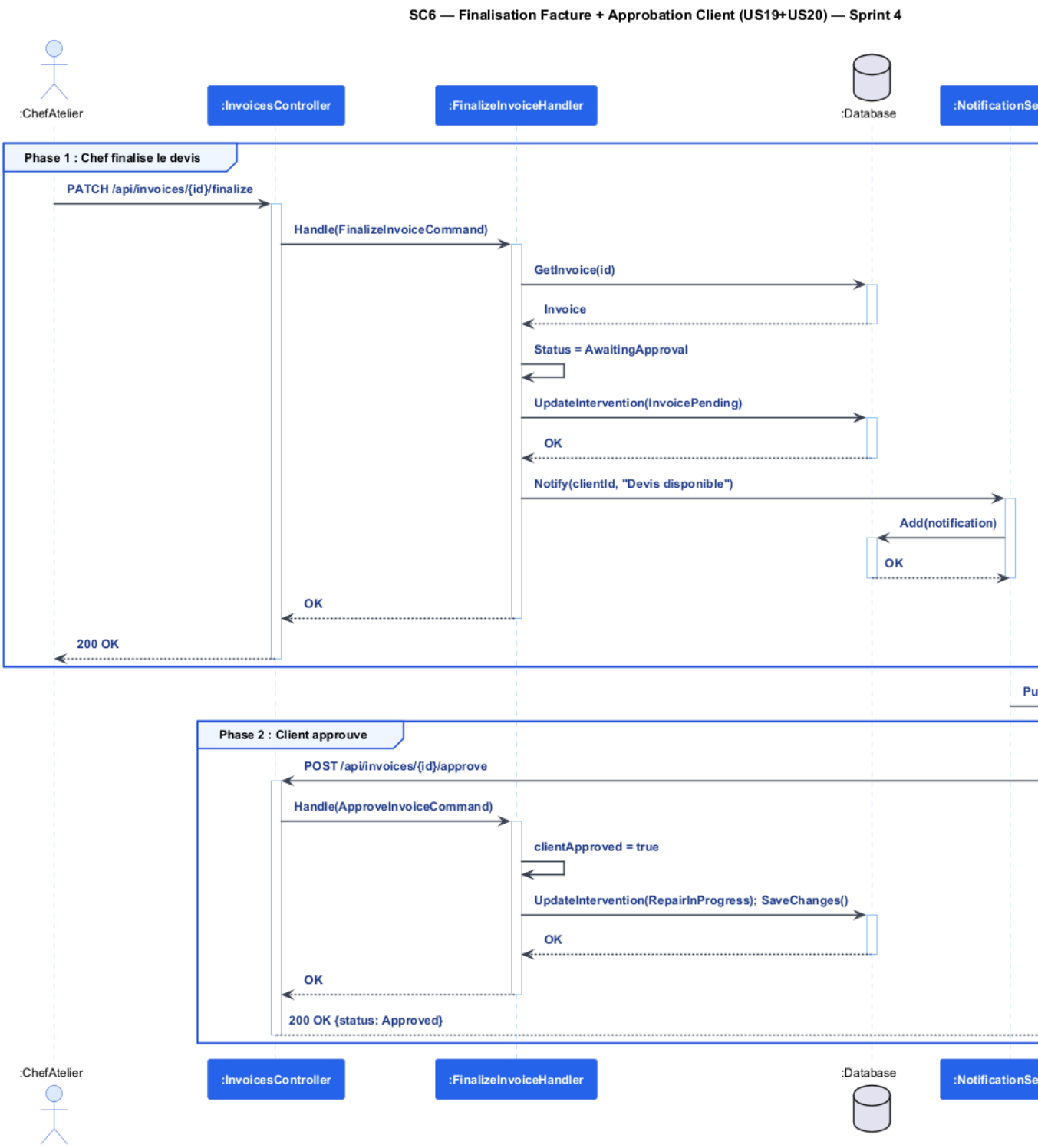


Figure 6.4. Diagramme de sÃ©quence SC6 — Finalisation et Approbation du Devis

6.4 Conclusion

Le Sprint 4 a rÃ©alisÃ© l'ensemble des opÃ©rations mÃ©tier de l'atelier, de l'affectation des mÃ©caniciens jusqu'Ã la clÃ´ture de l'intervention. Le *Intervention Lifecycle Tracker*, mis Ã jour automatiquement Ã chaque Ã©tape via MediatR, offre

une traÃ§abilitÃ© complÃ¨te. La gÃ©nÃ©ration de facture PDF (QuestPDF), les notifications la synchronisation SQLite → MySQL via FastAPI et l'application mobile Android complètent l'expérience utilisateur.

Conclusion Générale

Ce chapitre synthétise les réalisations du projet MecaRage et présente les perspectives d'évolution envisagées.

Le projet **MecaRage** représente une solution logicielle complète et cohérente pour la modernisation de la gestion des ateliers mécaniques. À travers quatre sprints successifs, nous avons construit une plateforme qui couvre l'intégralité du cycle de vie d'une intervention automobile : de la réception initiale des symptômes jusqu'au paiement final.

Bilan des réalisations

Sprint	Fonctionnalités	SP réalisés	US
Sprint 1	Authentification JWT, inscription, refresh token, profil	10	4
Sprint 2	Tenants, garages, personnel, véhicules, inventaire pièces	21	5
Sprint 3	Symptômes, diagnostic IA RAG, révision chef, rendez-vous	24	5
Sprint 4	Atelier, facture, approbation, réparation, paiement, PDF, mobile	62	13
	Total	**117 SP**	**27 US**

Table 6.2. Récapitulatif des réalisations par sprint

Les principaux apports techniques du projet sont :

— **Clean Architecture avec CQRS** : séparation des préoccupations, testabilité et maintenabilité à long terme via MediatR.
— **Service de diagnostic IA** : architecture RAG (BM25 + Ollama LLM local) produisant un pré-diagnostic structuré et contextualisé.
— **Intervention Lifecycle Tracker** : suivi transverse automatique sur 10 états, déclenché par les événements existants sans modifier le flux métier.

— **Gestion multi-tenant** : isolation totale des données de chaque entreprise via un filtre global EF Core (`HasQueryFilter`).
— **Application mobile Android (Java + SQLite)** : stockage local hors-ligne avec synchronisation périodique directe vers MySQL via un service FastAPI dédié.
— **Génération PDF (QuestPDF)** : factures professionnelles téléchargeables directement depuis l'interface client web et mobile.
— **Infrastructure conteneurisée** : Docker Compose supervisé par Prometheus et Grafana pour le monitoring en production.

Perspectives d'évolution

1. **Enrichissement de la base IA** : intégration de nouvelles marques et modèles de véhicules dans `knowledge_base.txt`, fine-tuning du modèle.
2. **Alertes de réapprovisionnement** : décrémenter automatiquement le stock pièces lors de la validation d'examen et alerter sous le seuil de réordre.
3. **Calendrier interactif** : vue mensuelle des rendez-vous via `@fullcalendar/angular` pour une meilleure planification.
4. **Export comptable** : export CSV/Excel des interventions clôturées sur une période pour intégration avec les outils comptables.
5. **Notation et avis client** : évaluation 5 étoiles + commentaire après la clôture d'une intervention.
6. **SignalR** : mise à jour en temps réel du tableau de bord web sans rechargement de page (actuellement polling).
7. **Audit Log** : enregistrement de chaque action sensible via un pipeline MediatR IPipelineBehavior pour la conformité et le débogage.

Bilan personnel

Ce projet nous a permis d'appliquer concrètement des patterns d'architecture logicielle modernes dans un contexte métier réel et contraignant (multi-tenant, multi-rôle, multi-plateforme). L'intégration de technologies hétérogènes — .NET, Angular, Python, Java, SQLite — dans une solution cohérente a constitué le principal défi et la principale source d'apprentissage.

Bibliographie

[1] Microsoft Corporation. *ASP.NET Core 9 Documentation*. Microsoft Docs, 2025. https://docs.microsoft.com/aspnet/core

[2] SQLite Consortium. *SQLite — Documentation Officielle*. SQLite, 2025. https://www.sqlite.org/docs.html

[3] FastAPI Team. *FastAPI — High Performance Python Web Framework*. Tiangolo, 2025. https://fastapi.tiangolo.com

[4] Ollama Team. *Ollama — Run Large Language Models Locally*. Ollama, 2025. https://ollama.com

[5] Angular Team. *Angular v21 Documentation*. Angular.io, 2025. https://angular.io/docs

[6] Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN 0-321-12521-5.

[7] Young, G. *CQRS Documents*. Greg Young, 2010. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

[8] QuestPDF Team. *QuestPDF — Modern .NET PDF Library*. QuestPDF, 2025. https://www.questpdf.com

[9] Robertson, S. & Robertson, J. *Mastering the Requirements Process*. 3e édition. Addison-Wesley, 2012. ISBN 0-321-81574-5.

[10] Pomelo Foundation. *Pomelo.EntityFrameworkCore.MySql — GitHub*. 2025. https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql

[11] Android Developers. *Android Java — Developer Guide*. Google, 2025. https://developer.android.com/guide